

WebSocket 동시 접속 성능 테스트

프로젝트: Pickers

목적: 리팩토링 후 WebSocket 서버의 동시 접속 처리 능력 및 메모리 안정성 검증

테스트 결과 요약

항목	결과
100명 동시 접속 시 Used Heap	53 MB
100명 5분 유지 후 Used Heap	41 MB (GC 톱니 패턴, 누수 없음)
전체 종료 + GC 후 Used Heap	37 MB (안정 범위 수렴)
CPU 사용률	0.2% ~ 1.0%
Live Threads	55 ~ 57개 (접속 수 무관)
브로드캐스팅 수신	100명 × 5분간 372,900건 수신 확인

1. 테스트 환경

항목	내용
서버	Spring Boot 3.3.6 + Java 17
WebSocket	Java-WebSocket 1.5.2
포트	9000 (내부 WebSocket 서버)
메시지	시뮬레이션 모드 브로드캐스팅 (1초 간격, 다중 종목)
클라이언트	Chrome 브라우저, JavaScript WebSocket API
측정 도구	VisualVM 2.1.10 (JVM Heap, CPU, Threads 모니터링)
테스트 페이지	test_websocket_v2.html (수신 카운터 + 동시 종료 기능)

리팩토링 후 추가된 기능이 모두 동작하는 상태에서 측정했다.

- 다중 종목 시뮬레이션 브로드캐스팅 동작 중
- Health Check 스케줄러 동작 중
- DB write-back 로직이 포함된 코드 베이스 기준으로 측정

2. 테스트 시나리오

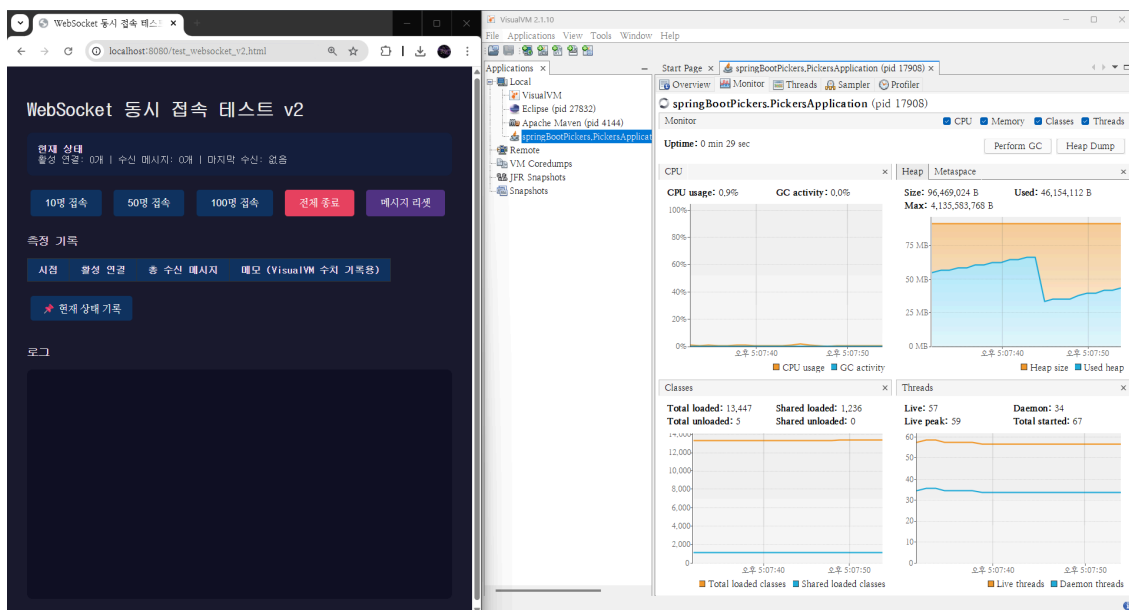
Step	내용
Step 0	서버 실행 후 기본 메모리 측정
Step 1	10명 동시 접속 → 30초 대기 → 수신 확인 → 측정
Step 2	전체 종료 → GC 대기 → 50명 접속 → 30초 대기 → 측정
Step 3	전체 종료 → GC 대기 → 100명 접속 → 30초 대기 → 측정
Step 4	100명 접속 상태 5분 유지 → Heap 추이 확인
Step 5	100명 동시 종료 → 종료 직후 상태 확인 → 추가 5분 대기 후 자연 GC 안정 수렴 확인

각 단계 사이에 전체 종료 → 30초 GC 대기 → 메시지 카운터 리셋 후 진행했다.

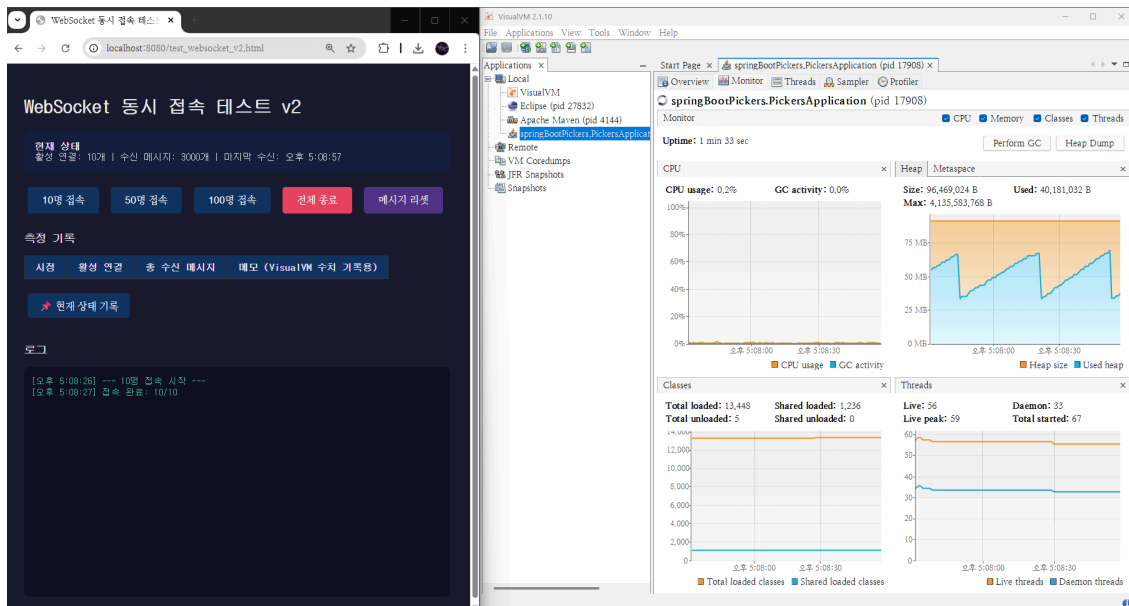
3. 측정 결과

3.1 메모리 및 리소스 사용량

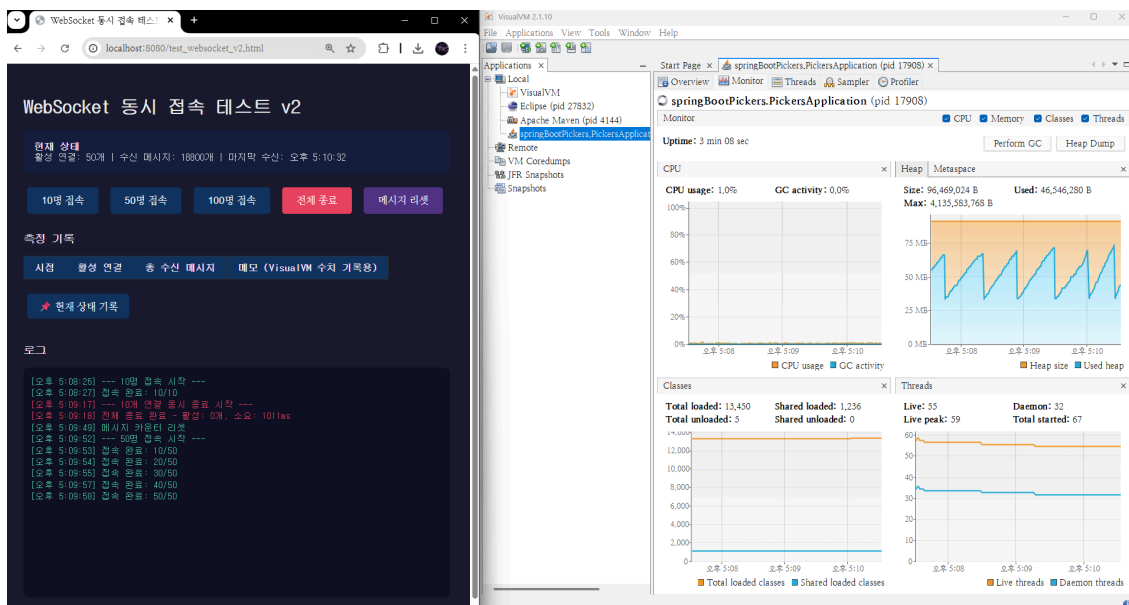
시점	Used Heap	Heap Size	CPU %	Live Threads	수신 메시지
기본 (0명)	44 MB	92 MB	0.9%	57	-
10명 접속	38 MB	92 MB	0.2%	56	3,000건
50명 접속	44 MB	92 MB	1.0%	55	18,800건
100명 접속	53 MB	92 MB	1.0%	55	45,300건
100명 5분 유지	41 MB	92 MB	1.0%	55	372,900건
전체 종료 직후	39 MB	92 MB	1.0%	55	-
GC 후 (5분 대기)	37 MB	92 MB	0.8%	55	-



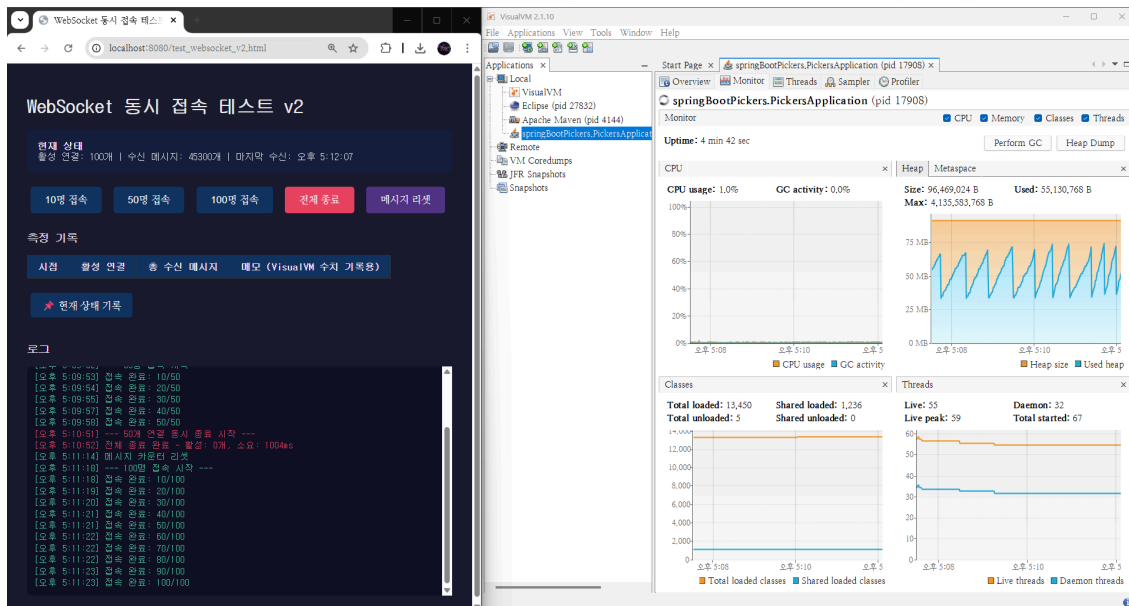
서버 실행 직후 — Used Heap 44MB, Live Threads 57



10명 접속 — Used Heap 38MB, 수신 메시지 3,000건



50명 접속 — Used Heap 44MB, 수신 메시지 18,800건



100명 접속 — Used Heap 53MB, 수신 메시지 45,300건

3.2 VisualVM 상세 데이터

기본 (서버 실행 직후):

Used: 46,154,112 B (44.0 MB)
 Size: 96,469,024 B (92.0 MB)
 Max: 4,135,583,768 B (3.85 GB)
 Live Threads: 57

10명 접속:

Used: 40,181,032 B (38.3 MB)
 Size: 96,469,024 B (92.0 MB)
 접속 완료: 10/10명
 수신 메시지: 3,000건

50명 접속:

Used: 46,546,280 B (44.4 MB)
 Size: 96,469,024 B (92.0 MB)
 접속 완료: 50/50명
 수신 메시지: 18,800건

100명 접속:

Used: 55,130,768 B (52.6 MB)
 Size: 96,469,024 B (92.0 MB)
 접속 완료: 100/100명
 수신 메시지: 45,300건

100명 5분 유지 후:

Used: 42,733,696 B (40.8 MB)
Size: 96,469,024 B (92.0 MB)
수신 메시지: 372,900건

전체 종료 + GC 후:

Used: 38,545,920 B (36.8 MB)
Size: 96,469,024 B (92.0 MB)

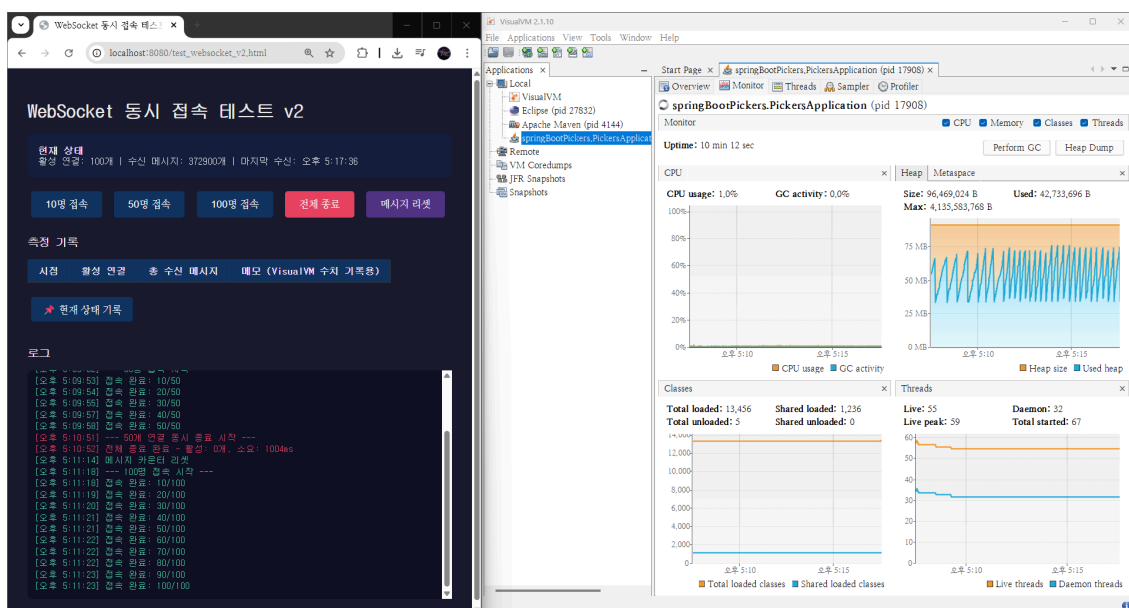
4. 메모리 안정성 검증

4.1 5분 유지 테스트

100명이 접속한 상태에서 5분간 브로드캐스팅을 계속 수신했다.

VisualVM Heap 그래프에서 GC가 반복적으로 메모리를 회수하는 톱니 패턴이 확인된다.

Used Heap이 무한 우상향하지 않으며, GC 후 안정 구간으로 반복 복귀한다.

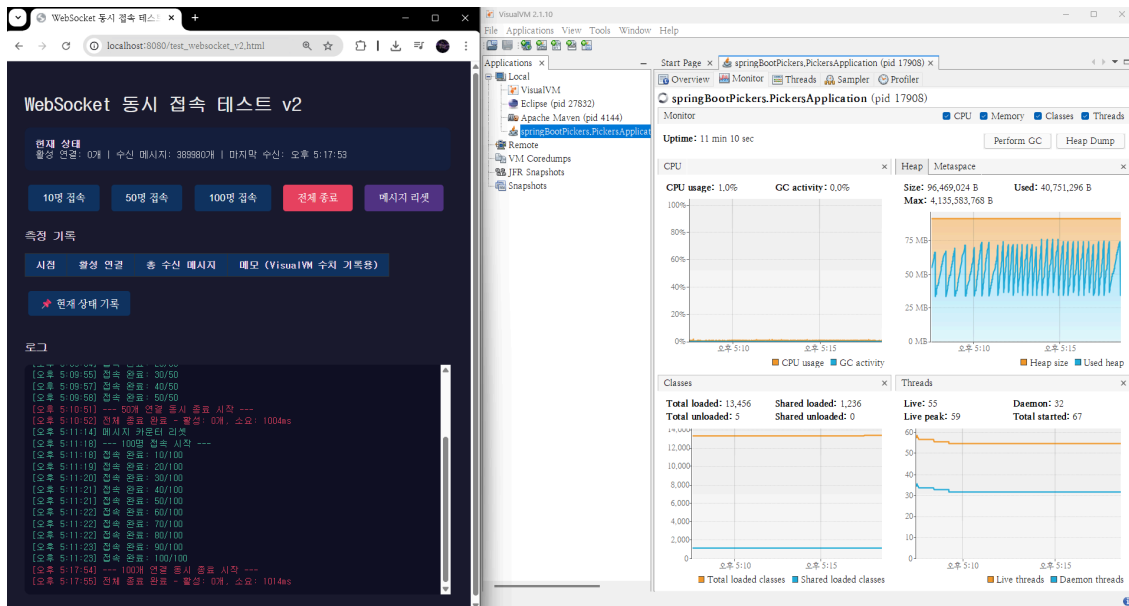


100명 5분 유지 — Heap 톱니 패턴 확인, 수신 메시지 372,900건

4.2 동시 종료 + GC 회수

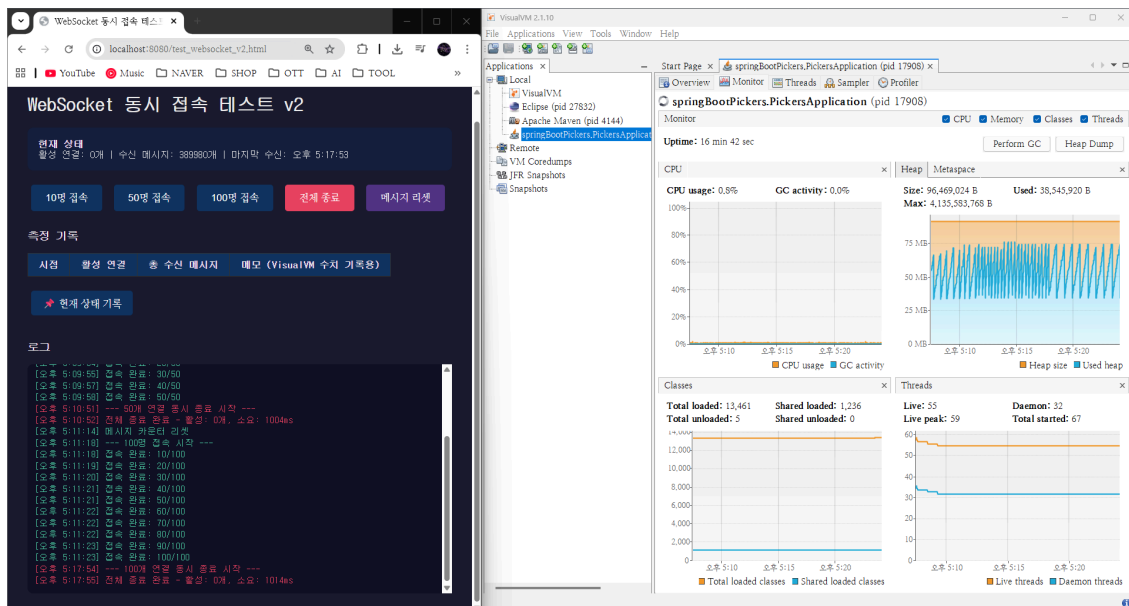
100명 전원을 동시에 종료했다.

테스트 페이지 로그에서 "100개 연결 동시 종료 시작 → 종료 요청 후 약 1초 뒤 활성 연결 0개 확인했다."



전체 종료 직후 — 활성 연결 0개, 종료 소요 약 1초

종료 후 5분 대기하여 GC가 자연 동작하도록 했다.
Used Heap이 37 MB로 수렴했으며, 기본 상태(44 MB)보다 낮은 수준이다.
이는 JVM 워업 초기 클래스 로딩 등이 이후 정리된 것으로, 정상 동작이다.



GC 후 메모리 수렴 — Used Heap 37MB

4.3 GC 회수 검증

항목	수치
100명 접속 시	53 MB
전체 종료 + GC 후	37 MB
회수량	16 MB
안정 범위 수렴 여부	O

5. 기술적 분석

5.1 ConcurrentHashMap의 효과

```
private final Set<WebSocket> connections = ConcurrentHashMap.newKeySet();

@Override
public void onClose(WebSocket conn, int code, String reason, boolean remote) {
    connections.remove(conn);
    log.info("WS 클라이언트 종료: {}", conn.getRemoteSocketAddress());
}
```

연결 해제 시 `onClose()` 에서 `connections.remove(conn)` 이 호출되어 Set에서 제거되고, 참조가 끊긴 WebSocket 객체는 GC 대상이 된다. 100명 동시 종료 후 활성 연결 0, GC 후 메모리 수렴으로 이 메커니즘이 정상 동작함을 확인했다.

5.2 스레드 안정성

접속자 수	Live Threads
0명	57
10명	56
50명	55
100명	55

접속자가 증가해도 Live Threads가 55~57로 거의 변동이 없다. WebSocket 연결이 별도 스레드를 생성하지 않고, NIO 기반으로 단일 스레드에서 다중 연결을 처리하기 때문이다. 접속자 수에 비례해 스레드가 증가하지 않으므로, 스레드 고갈 위험이 없다.

5.3 CPU 사용률

접속자 수	CPU %
0명	0.9%
10명	0.2%
50명	1.0%
100명	1.0%

100명에게 1초 간격으로 다중 종목 시세를 브로드캐스팅해도 CPU 1% 수준이다. 브로드캐스팅은 `connections` Set을 순회하며 `conn.send()` 를 호출하는 O(n) 연산이지만, JSON 문자열 전송 자체가 가벼워서 100명 수준에서는 부하가 미미하다.

6. 판정 기준 및 결과

판정 기준	결과
100명 5분 유지 중 Used Heap 무한 우상향 없음	✓ 통과 — 톱니 패턴 확인
전체 종료 후 활성 연결 0 확인	✓ 통과 — 종료 요청 후 1초 뒤 확인 시 활성 연결 0개
종료 후 GC 이후 안정 범위 수렴	✓ 통과 — 37 MB 수렴
Live Threads / CPU 비정상 유지 없음	✓ 통과 — 55개 / 0.8%

7. 결론

리팩토링 후(다중 종목 처리, DB write-back, Health Check 추가) WebSocket 서버에 100명이 동시 접속하여 5분간 실시간 시세를 수신한 결과, 메모리 누수 없이 안정적으로 동작했다.

ConcurrentHashMap 기반 세션 관리와 `onClose()` 자동 제거가 정상 작동하여, 100명 동시 종료 후 GC를 통해 메모리가 안정 범위로 수렴했다.

접속자 증가에도 스레드 수(55개)와 CPU(1%)가 안정적으로 유지되어, 현재 구조에서 100명 동시 접속은 서버에 유의미한 부하를 주지 않는다.